

README

Публичный многоязычный сайт-лендинг для платформы ProjectOO.

Production:

- <https://organizationoffice.com>
- <https://www.organizationoffice.com>
- <https://organization2025.com>

Vue 3 / TypeScript / Vite SPA. Бэкенд — [organizationoffice/api](#), используется только endpoint обратной связи `/api/public/send_feedback` .

Что внутри

- **10 страниц** (landing, описания фич, тарифы, мобилка, политика конфиденциальности, оферта)
- **8 языков** (en, ua, de, pl, fr, it, bg, ar) через vue-i18n, переключение через URL prefix
- **PWA** — installable, standalone display mode, maskable иконки
- **3 формы** (call-back, consultation request, admin access) — отправка через `/api/public/send_feedback`
- **Анимации** — IntersectionObserver scroll-reveals + animated count-up для статистики
- **Zapier chatbot** виджет в `<body>` для live-чата

Технологический стек

Слой	Технология
Framework	Vue 3.5 (Composition API)
Язык	TypeScript 5.9
Сборщик	Vite 6
Router	vue-router 4 (language-prefix mode)
i18n	vue-i18n 11
HTTP	axios 1.12
Styles	SCSS (sass 1.93) + Roboto Condensed
XML parsing	fast-xml-parser (для <code>/public/xml/rgo.xml</code>)
Lint	ESLint 9 + @typescript-eslint + eslint-plugin-vue
Chat	Zapier Interfaces (embed)

Быстрый старт

```
git clone git@github.com:organizationoffice/web.git
cd web
npm ci
npm run dev
```

Сборка:

```
npm run build      # vue-tsc + vite build → dist/
npm run forcebuild # без type-check, для срочных релизов
npm run preview    # локальная проверка production-сборки
```

Подробнее: [docs/SETUP.md](#).

Документация

Документ	Описание
docs/SPECIFICATION.md	Функциональная спецификация (структура + страницы + roadmap)
docs/specs/	Per-module specs (pages, i18n, forms, seo-pwa...)
docs/ARCHITECTURE.md	Архитектура, layout, routing strategy
docs/SETUP.md	Локальная настройка, сборка, деплой
docs/CONTENT-GUIDE.md	Как редактировать тексты, добавлять страницы, языки, изображения

Структура

```

web/
├── src/
│   ├── main.ts                # Vue app entry
│   ├── App.vue                # Layout: Header + RouterView + Footer + Toast
│   ├── router/index.ts        # /:lang/* routes + guard
│   ├── views/                 # 10 страниц
│   │   ├── Index.vue          # Landing
│   │   ├── Control.vue, How.vue, Go.vue, Meeting.vue
│   │   ├── MobileApp.vue, Tariffs.vue, Admin.vue
│   │   └── PrivacyPolicy.vue, PublicOffer.vue
│   ├── components/            # Header.vue, Footer.vue
│   ├── service/
│   │   ├── i18n.ts            # 8-language setup, SUPPORT_LOCALES
│   │   ├── global.ts          # sendMail, toast, scrollAnimate, countUp
│   │   ├── constants.ts       # APPS, SOCIALS, PHONES, EMAILS
│   │   └── global-properties.d.ts
│   ├── lang/                  # en/ua/de/pl/fr/it/bg/ar.json
│   └── assets/
│       ├── style.scss          # Global colors, layout utils
│       ├── _fonts.scss
│       └── fonts/RobotoCondensed/
├── public/
│   ├── img/                   # 20 PNG: hero, features, team (7 portraits)
│   ├── svg/                   # 53 SVG: logo, social, UI icons, decor
│   ├── xml/rgo.xml            # Public org registry data
│   ├── favicon*, *.png        # PWA + favicons
│   └── site.webmanifest        # PWA config
├── index.html                 # + Zapier chatbot widget
├── vite.config.ts             # /api proxy в dev
├── tsconfig.json              # + tsconfig.app.json, tsconfig.node.json
├── eslint.config.ts
└── .github/workflows/deploy.yml # CI/CD

```

CI/CD

- Trigger: push в `main` или manual `workflow_dispatch`
- Runner: self-hosted `p2-web` на p2 (третий рядом с api и webadmin)
- Шаги: `npm ci` → `npm run build` → `rsync dist/` → `/var/www/organization2025.com/` → optional Cloudflare purge

Подробнее: [.github/workflows/deploy.yml](#) и [docs/SETUP.md](#).

Связь с backend

Минимальная — только один endpoint:

- `POST /api/public/send_feedback` — отправка форм обратной связи (call-back, consultation, admin access)

Все вызовы идут с относительным path (`/api/...`); в dev проксируется через vite на `organization2025.com` , в prod nginx сам разруливает.

Лицензия

Проприетарный код. © OrganizationOffice.

Architecture

Vue 3 SPA — публичный сайт-лендинг. Минималистичная архитектура: 10 страниц, общий layout, language-prefix routing.

Высокоуровневая диаграмма



Принципы:

- **No state** — это маркетинговый сайт, нет логина, нет авторизации, нет localStorage (кроме потенциальной памяти языка через URL)
- **Language as URL** — `/:lang/page` единственный источник правды о выбранном языке
- **Single page bundle** — все 10 страниц в одном JS bundle (~150 KB gzip)
- **Static assets из public/** — раздаются nginx напрямую с диска
- **No SSR** — pure client-side rendering, SEO компенсируется PWA + grew sitemap (если настроен)

Слои

1. Entry — `src/main.ts`

Регистрирует плагины:

- `vue-router` — роутер
- `vue-i18n` — переводы
- Global property `$sendMail` (alias для `sendMail()` из `services/global.ts`)
- Импорт `assets/style.scss` (глобальные стили)

2. Root — `src/App.vue`

Структура:

```
<page-container>
  <Header />
  <RouterView />
  <Footer />
  <toast />
</page-container>
```

Toast живёт глобально (fixed at top, slide-in transition, auto-hide 5s, close button).

3. Layout components — `src/components/`

`Header.vue`

- Логотип (left)
- Nav menu: 2 секции (`nav1` — Board / Members / Meetings; `nav2` — Tariffs / Contacts / Management)
- Language switcher (dropdown с флагами 8 локалей)
- Mobile hamburger menu на узких экранах
- Декоративный triangle (SVG)

`Footer.vue`

- Контакты: телефоны, email, working hours
- Социальные сети (Instagram, YouTube, TikTok, Telegram, LinkedIn) — с iconified SVG

- Telegram QR-код для быстрого подключения
- Форма callback (mini-form: имя + телефон → `sendMail()`)

4. Pages — `src/views/`

10 .vue файлов, каждый — самостоятельная page. См. [specs/pages.md](#).

5. Services — `src/service/`

- `i18n.ts` — создаёт `vue-i18n` instance с 8 локалями, exports `SUPPORT_LOCALES = ['en', 'ua', 'de', 'pl', 'fr', 'it', 'bg', 'ar']` . Fallback = `en` .
- `global.ts` — глобальные хелперы:
 - `sendMail(payload)` — POST на `/api/public/send_feedback` (используется всеми формами)
 - `showToast(message, type?)` — показать toast
 - `scrollAnimate(selector)` — IntersectionObserver-based reveal анимации
 - `animateCountUp(element, target)` — animated counter (для stats блоков на лендинге)
- `constants.ts` — конфиг:
 - `APPS` — App Store / Google Play ссылки
 - `SOCIALS` — массив соц.сетей с URL и иконками
 - `PHONES` , `EMAILS` — контакты
- `global-properties.d.ts` — TypeScript-объявления глобальных свойств (`$sendMail` , `$router` , `$route`)

6. Translations — `src/lang/`

8 JSON-словарей, иерархические ключи:

```
{
  "header": { "nav1": { "control": "Board", ... } },
  "footer": { "callback": "Request a callback", ... },
  "index": { "s_1": { "title": "...", "s_2": {...} },
  "control": { "s_1": {...}, "s_4_1": {...}, ... },
  "tariffs": {...},
  "email": { "success": "...", "error": "..." }
}
```

Ключи именуются как `s_N` (section N) или `s_N_M` (sub-section M of section N) для page-specific блоков.

7. Assets — `src/assets/`

- `style.scss` — главный стиль. Палитра:

```
$bg-main:      #DBD6DE; // light purple background
$bg-secondary: white;
$bg-2:         #2E80B7; // blue accent
$bg-3:         black;
$bg-4..$bg-8:  pastel variations
$text-main, $text-light, $text-accent, ...
```

- utility classes (flex, spacing, animations) + global resets
- `_fonts.scss` — `@font-face` для Roboto Condensed (Regular/Medium/Bold)
- `fonts/RobotoCondensed/` — TTF файлы

Routing strategy: language prefix

Все routes начинаются с `:lang` параметра:

```
/en          → Index
/en/control   → Control
/en/mobile-app → MobileApp
...
/ua/tariffs   → Tariffs (ua language)
/de/admin     → Admin (de language)
```

Navigation guard

```
router.beforeEach((to, from, next) => {
  const lang = to.params.lang as string
  if (!SUPPORT_LOCALES.includes(lang)) {
    next({ path: `/en${to.path.replace(`/${lang}`, '')}` })
  } else {
    i18n.global.locale.value = lang
    next()
  }
})
```

То есть `/ru/control` (неподдерживаемый) → redirect на `/en/control`.

Scroll behavior

- При навигации по hash (`#section`): smooth scroll к якорю
- При back navigation: восстанавливается сохранённая позиция

State management

Нет — нет Pinia, нет Vuex, нет даже reactive global store.

- Язык — в URL
- Toast state — в App.vue local state

- Form state — в локальном state соответствующих компонентов

Это **намеренно** — для маркетингового сайта без сессий это не нужно.

i18n

- 8 локалей, переключение через URL prefix
- `vue-i18n@11` composition mode
- Translation tables — `src/lang/*.json`, total ~800 KB JSON но грузятся at-once (есть пространство для lazy-load)

Подробнее: [specs/i18n.md](https://projectoo.com/specs/i18n.md).

API integration

Минимальная — только формы:

- `POST /api/public/send_feedback { subject, body }` — отправка форм
- В body — HTML-формат с заполненными данными формы
- Backend пересылает на `support@organizationoffice.com` через Mailgun

Подробнее: [specs/api-integration.md](https://projectoo.com/specs/api-integration.md).

Animations

Используются 2 техники:

1. Scroll-triggered reveal

```
scrollAnimate('.fade-in')
// Под капотом:
const obs = new IntersectionObserver((entries) => {
  entries.forEach(e => e.isIntersecting && e.target.classList.add('visible'))
})
```

CSS:

```
.fade-in { opacity: 0; transition: opacity 0.6s; }
.fade-in.visible { opacity: 1; }
```

2. Animated count-up для статистики

```
animateCountUp(element, targetNumber)
// requestAnimationFrame loop: 0 → target за 2s
```

Используется на лендинге для блоков типа "120+ организаций", "5000+ пользователей".

PWA

- `site.webmanifest` — installable, standalone mode, maskable icons
- Service worker — **НЕ настроен** (потенциальный roadmap)
- Offline mode — отсутствует

SEO

- Title в `index.html`
- Google site verification meta
- Lang attribute (`<html lang="ua">` — захардкожено, по-хорошему должно быть dynamic per current locale)
- Sitemap / robots — нужно проверить, генерируется ли при build (по умолчанию vite не делает)
- RSS / sitemap — кандидаты на roadmap

Известные риски / архитектурные заметки

Риск	Где	Митигация
<code><html lang="ua"></code> хардкодом	<code>index.html</code>	Менять динамически на основе текущего <code>i18n</code> locale (для SEO и accessibility)
Все 8 locale JSON грузятся at-once	<code>services/i18n.ts</code>	Lazy-load + dynamic import — <code>import('./lang/' + lang + '.json')</code>
<code>rgo.xml</code> ~ непонятного размера	<code>public/xml/</code>	Не загружается автоматически — проверить, используется ли вообще
Нет SEO meta-тегов per page	<code>views/*</code>	Использовать <code>@unhead/vue</code> или <code>vue-meta</code> для динамических title/description per route
Нет service worker	—	Workbox для offline-кеша и push-notifications (опц.)
<code>tsconfig.app.json</code> без strict	<code>tsconfig</code>	Постепенно включать strict mode
Нет тестов	—	Vitest + Vue Test Utils
Zapier chatbot в <code><body></code>	<code>index.html</code>	Загружается синхронно при первом рендере → добавляет TTI; можно lazy-load или вынести в env-flag

Setup & Deployment

Требования

Инструмент	Версия	Зачем
Node.js	20.11+	runtime + npm
npm	10+	зависимости
Git	2.40+	clone

macOS

```
brew install node@22
```

Linux

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -  
sudo apt install -y nodejs
```

Клонирование и установка

```
git clone git@github.com:organizationoffice/web.git  
cd web  
npm ci    # exact install из package-lock.json
```

Запуск в dev-режиме

```
npm run dev
```

Vite dev server:

- HMR (мгновенная пересборка)
- Прокси `/api/*` → `https://organization2025.com/api/*`

Для разработки против локального backend измени `vite.config.ts` :

```
server: {
  proxy: {
    '/api': { target: 'http://localhost:8080', changeOrigin: true }
  }
}
```

Сборка

```
npm run build
```

```
npm run forcebuild
```

```
npm run preview
```

Типичный размер `dist/` :

- HTML + JS + CSS: ~1-2 MB pre-gzip, ~300-500 KB после
- Все 8 lang JSON в bundle: ~200 KB
- Images / fonts из public/: копируются как есть

Линт

```
npx eslint src --ext .ts,.vue
npx eslint src --ext .ts,.vue --fix
```

Roadmap: добавить "lint" script в package.json .

Деплой

Production target

Раздаётся через nginx из `/var/www/organization2025.com/` на p2 (93.115.21.210).

Один комплект файлов обслуживает 3 server_name:

- organization2025.com
- organizationoffice.com
- www.organizationoffice.com

nginx-конфиги:

- `/etc/nginx/conf.d/organization2025.com.conf`
- `/etc/nginx/conf.d/organizationoffice.com.conf`

Оба указывают `root /var/www/organization2025.com;` .

GitHub Actions (текущий способ)

`.github/workflows/deploy.yml` — auto на push в `main` , plus manual `workflow_dispatch` .

Шаги:

1. Checkout
2. Setup Node 22 + npm cache
3. `npm ci`
4. `npm run build`
5. `rsync dist/ → /var/www/organization2025.com/`
6. (опц.) Cloudflare cache purge — если установлены секреты `CF_ZONE` , `CF_EMAIL` , `CF_AUTH_KEY`

Runner — self-hosted `p2-web` на p2 (третий рядом с api и webadmin).

Required GitHub Secrets

Никаких — деплой self-hosted, права настроены на сервере.

Опционально:

Secret	Значение
<code>CF_ZONE</code>	Cloudflare Zone ID
<code>CF_EMAIL</code>	Cloudflare account email
<code>CF_AUTH_KEY</code>	Cloudflare Global API Key

Manual deploy (запасной путь)

```
npm ci && npm run build
rsync -avz --delete dist/ p2:/var/www/organization2025.com/
```

```
curl -X POST "https://api.cloudflare.com/client/v4/zones/$CF_ZONE/purge_cache" \
-H "X-Auth-Email: $CF_EMAIL" \
-H "X-Auth-Key: $CF_AUTH_KEY" \
-H "Content-Type: application/json" \
--data '{"purge_everything":true}'
```

nginx **не нужно** рестартить — статика, файлы подменяются на лету.

Rollback

```
gh workflow run deploy.yml --ref <sha-or-tag>
```

Roadmap: хранить N последних версий + симлинк-swap для instant rollback.

nginx config (фрагмент)

```
server {
    listen 443 ssl http2;
    server_name organization2025.com organizationoffice.com www.organizationoffice.com;

    root /var/www/organization2025.com;

    # SPA fallback – все неизвестные пути → index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Static files из public/
    location /img/    { try_files $uri =404; }
    location /svg/    { try_files $uri =404; }
    location /xml/    { try_files $uri =404; }
    location /favicon.svg { try_files $uri =404; }
    # ... остальная статика

    # API proxy к backend
    location /api/ {
        proxy_pass http://127.0.0.1:8080/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Diia webhook (для organizationoffice.com)
    location = /diia-webhook {
        proxy_pass http://127.0.0.1:8080/diia/documents-receiver;
    }
}
```

Локализация

Файлы переводов — `src/lang/<locale>.json` . Поддерживается 8 языков (`en` , `ua` , `de` , `pl` , `fr` , `it` , `bg` , `ar`).

Подробнее: [CONTENT-GUIDE.md](#).

Частые проблемы

Симптом	Причина	Решение
<code>npm ci</code> падает на <code>vue-tsc</code>	старый Node	поставить 20.11+
Refresh страницы → 404	nginx без <code>try_files</code>	добавить <code>try_files \$uri \$uri/ /index.html;</code>
После деплоя старая версия	Cloudflare кеш	<code>purge cache</code> или hard reload
Zapier chatbot не загружается	блокировщик / wrong chatbot-id	проверить ID в <code>index.html</code>
Форма callback не отправляется	backend (api) недоступен	проверить <code>POST /api/public/send_feedback</code>
Сборка падает с TS-ошибками	<code>vue-tsc -b</code> нашёл несоответствия	<code>npm run forcebuild</code> или исправить типы
<code>/ua/control</code> → редирект на <code>/en/control</code>	<code>ua</code> не в <code>SUPPORT_LOCALES</code>	проверить <code>service/i18n.ts</code> — он точно там

Content Guide

Гайд для контент-менеджеров и переводчиков: как менять тексты, добавлять страницы, языки, изображения.

Структура контента

```
src/
├── lang/
│   ├── en.json
│   ├── ua.json
│   ├── de.json
│   ├── pl.json
│   ├── fr.json
│   ├── it.json
│   ├── bg.json
│   └── ar.json
└── views/
```

Все переводы — здесь.
Структура одинаковая, переводятся значения.

10 страниц; HTML/template + ссылки на `$t()` ключи

Изменение текста на странице

Все тексты вынесены в `src/lang/<locale>.json` и используются через `$t('key.path')`.

Пример: на главной странице нужно поменять заголовок hero-секции.

1. Открыть `src/views/Index.vue`, найти место:

```
<h1>{{ $t('index.s_1.title') }}</h1>
```

2. Открыть `src/lang/en.json`, найти ключ:

```
"index": {
  "s_1": {
    "title": "Smart platform for ...",
    ...
  }
}
```

3. Поменять значение `title`.
4. **Повторить для всех 8 языков** — иначе пользователи non-English locales увидят fallback с английским текстом.

Tip: чтобы найти все языки разом — `grep -r "s_1.title" src/lang/`.

Добавление новой страницы

Например, страница "About Us" (`/:lang/about`).

1. Создать компонент

`src/views/About.vue` :

```
<template>
  <section class="about-page">
    <h1>{{ $t('about.title') }}</h1>
    <p>{{ $t('about.description') }}</p>
  </section>
</template>

<script setup lang="ts">
  // логика, если нужна
</script>

<style scoped lang="scss">
.about-page {
  padding: 60px 24px;
  // ...
}
</style>
```

2. Добавить route

`src/router/index.ts` :

```
import About from '@views/About.vue'

routes: [
  // ...
  { path: '/:lang/about', component: About, name: 'about' }
]
```

3. Добавить переводы

Во все 8 файлов `src/lang/*.json` добавить:

```
{
  "about": {
    "title": "About Us",
    "description": "..."
  }
}
```

4. Добавить пункт в навигацию (если нужно)

`src/components/Header.vue` — найти секцию `nav1` или `nav2` :

```
<router-link :to="`/${i18n.locale}/about`">
  {{ $t('header.nav1.about') }}
</router-link>
```

И в `lang/*.json` под `header.nav1` добавить `"about": "About"` .

Добавление нового языка

Например, испанский (`es`).

1. Создать словарь

```
cp src/lang/en.json src/lang/es.json
```

Открыть `src/lang/es.json` и перевести все значения.

2. Добавить в `i18n.ts`

`src/service/i18n.ts` :

```
import es from '@lang/es.json'

export const SUPPORT_LOCALES = ['en', 'ua', 'de', 'pl', 'fr', 'it', 'bg', 'ar', 'es']

const i18n = createI18n({
  // ...
  messages: { en, ua, de, pl, fr, it, bg, ar, es }
})
```

3. Добавить в language switcher

`src/components/Header.vue` — в dropdown:

```
<option value="es">🇪🇸 Español</option>
```

4. (опц.) Добавить иконку флага

Если используется SVG-флаг, положить в `public/svg/flags/` и добавить ссылку.

Изменение изображений

Hero / page images (`public/img/`)

Имена соответствуют странице/секции:

- `index_1_1.png` — первая большая картинка (`s_1`) на Index
- `index_4_1.png` — картинка для секции 4
- `control_1_1.png` , `meeting_1_1.png` , `mobile-app_1_1.png` — для соответствующих страниц

Чтобы заменить:

1. Положить новый файл с тем же именем (формат PNG / WebP)
2. Размер: рекомендуется 1920×1080 или меньше, оптимизированный через TinyPNG
3. Hard reload в браузере (`Cmd+Shift+R`)

Иконки и декор (`public/svg/`)

53 SVG. Названия описательные:

- `logo.svg` , `logo-sm.svg` — логотип
- `in.svg` / `in-w.svg` — Instagram (dark / white)
- `triangle.svg` , `bg-1.svg` — декор

Чтобы добавить новую иконку: положить SVG (минимизированный) в `public/svg/` и ссылаться через `` .

Team photos (`public/img/team_N.png`)

Сейчас 7 фото (`team_1.png` ... `team_7.png`).

Чтобы добавить 8-го члена команды:

1. Положить `team_8.png`
2. Открыть страницу, где они показываются (вероятно `Index.vue` или отдельная), и добавить markup для `team_8`

PWA / favicons

В `public/` :

- `favicon.ico` , `favicon.svg` , `favicon-96x96.png`
- `apple-touch-icon.png` (180×180 для iOS home screen)
- `web-app-manifest-192x192.png` , `web-app-manifest-512x512.png` (PWA installable icon)
- `site.webmanifest` — конфиг PWA (имя, цвета)

При смене брендинга / логотипа — регенерировать **все 5** размеров (например, через [realfaviconsgenerator.net](https://realfavicons.com/)).

Меняем контакты, соц.сети, ссылки на стор

`src/service/constants.ts` :

```
export const PHONES = ['+38 (XXX) XXX-XX-XX']
export const EMAILS = ['support@organizationoffice.com']

export const APPS = [
  { name: 'App Store', url: 'https://apps.apple.com/...', icon: '/svg/apple.svg' },
  { name: 'Google Play', url: 'https://play.google.com/...', icon: '/svg/googleplay.svg' }
]

export const SOCIALS = [
  { name: 'Instagram', url: 'https://...', icon: '/svg/in.svg' },
  // ...
]
```

После правок сайт автоматически подхватит новые значения в Header / Footer / соответствующих секциях.

Меняем цвета / шрифты

src/assets/style.scss :

```
$bg-main:      #DBD6DE; // главный фон страницы
$bg-secondary: white;  // карточки, secondary блоки
$bg-2:         #2E80B7; // акцент (СТА-кнопки, ссылки)
$bg-3:         black;
$bg-4..$bg-8:  #...;    // дополнительные акценты
$text-main:    #222;
$text-light:   #888;
$text-accent:  #2E80B7;
```

Меняешь — пересобираешь, и весь сайт обновляется (через SCSS variables).

Для смены шрифта:

1. Положить новые .ttf / .woff2 в src/assets/fonts/
2. Обновить src/assets/_fonts.scss (заменить @font-face правила)
3. Обновить style.scss : font-family: 'NewFont', sans-serif;

Меняем chatbot

В index.html найти строку:

```
<zapier-interfaces-chatbot-embed chatbot-id="cmftsrv4001a10558q2960z1"></zapier-interfaces-
```

Заменить chatbot-id на нужный (получается в [Zapier Interfaces](#)).

Превью изменений локально

```
npm run dev
```

После build:

```
npm run build  
npm run preview
```

Чек-лист перед коммитом

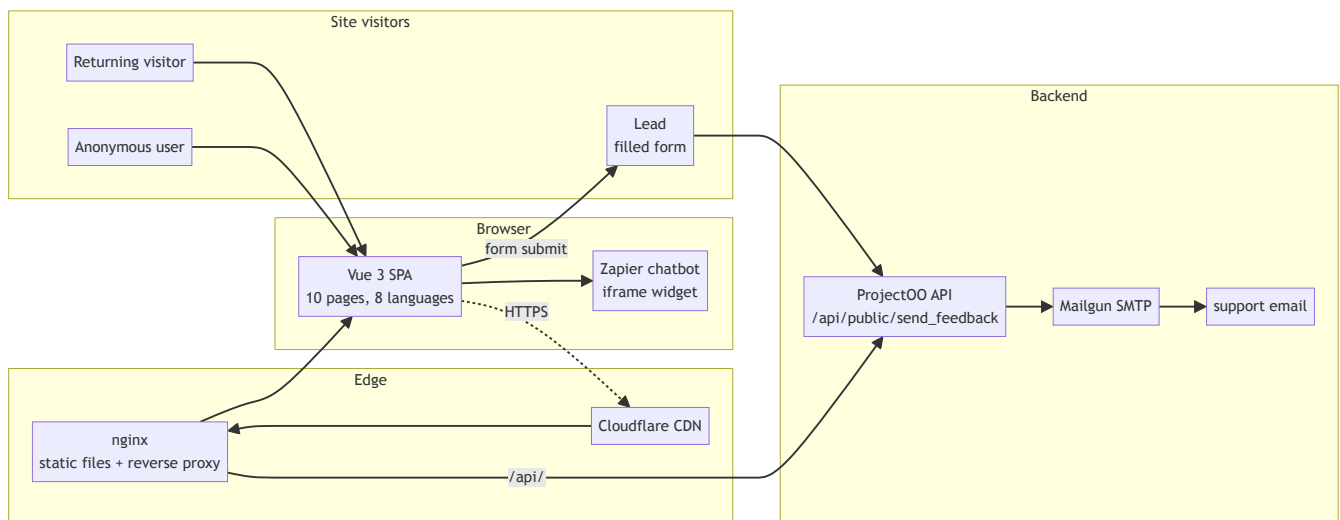
- ☐ Текст изменён во всех 8 lang файлах (или явно решено — только в каком-то)
- ☐ Изображения сжаты (PNG/WebP < 200 KB)
- ☐ SVG минимизированы (через [SVGOMG](#))
- ☐ `npm run build` проходит без ошибок
- ☐ `npm run preview` — визуально проверить
- ☐ Проверить адаптивность на mobile-эмуляторе (DevTools)

Functional Specification

Функциональная спецификация публичного сайта-лендинга. Описывает систему с точки зрения **бизнес-функциональности**: какие страницы, какой контент, какие формы.

Формат — box-diagrams + descriptions + roadmap (повторяет стиль спецификаций api и webadmin).

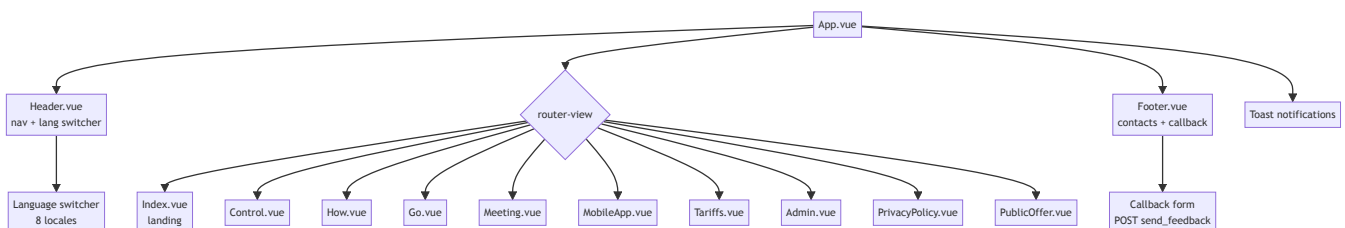
1. System Main Structure



Принципы:

- **No login**: всё доступно без аутентификации
- **No state**: нет cookies / localStorage для UI state
- **Language via URL**: `/: lang/page` — единственный источник правды
- **Static-first**: всё разруливает nginx, backend нужен только для отправки форм
- **Multi-domain**: один build → `organizationoffice.com` + `organization2025.com`

2. UI Structure



3. Page Catalogue

Page	URL	Назначение
Index	<code>/:lang</code>	Landing c hero, consultation form, stats, features, testimonials, CTA
Control	<code>/:lang/control</code>	Описание возможностей управления / Board panel
How	<code>/:lang/how</code>	Step-by-step гайд "How it works"
Go	<code>/:lang/go</code>	Onboarding / Getting started
Meeting	<code>/:lang/meeting</code>	Описание возможностей online-встреч
MobileApp	<code>/:lang/mobile-app</code>	Промо мобильного приложения (iOS + Android, QR-коды)
Tariffs	<code>/:lang/tariffs</code>	Тарифные планы / pricing
Admin	<code>/:lang/admin</code>	Описание админ-возможностей (для потенциальных орг-адинов)
PrivacyPolicy	<code>/:lang/privacy-policy</code>	Политика конфиденциальности (юридический текст)
PublicOffer	<code>/:lang/public-offer</code>	Публичная оферта / Terms of Service

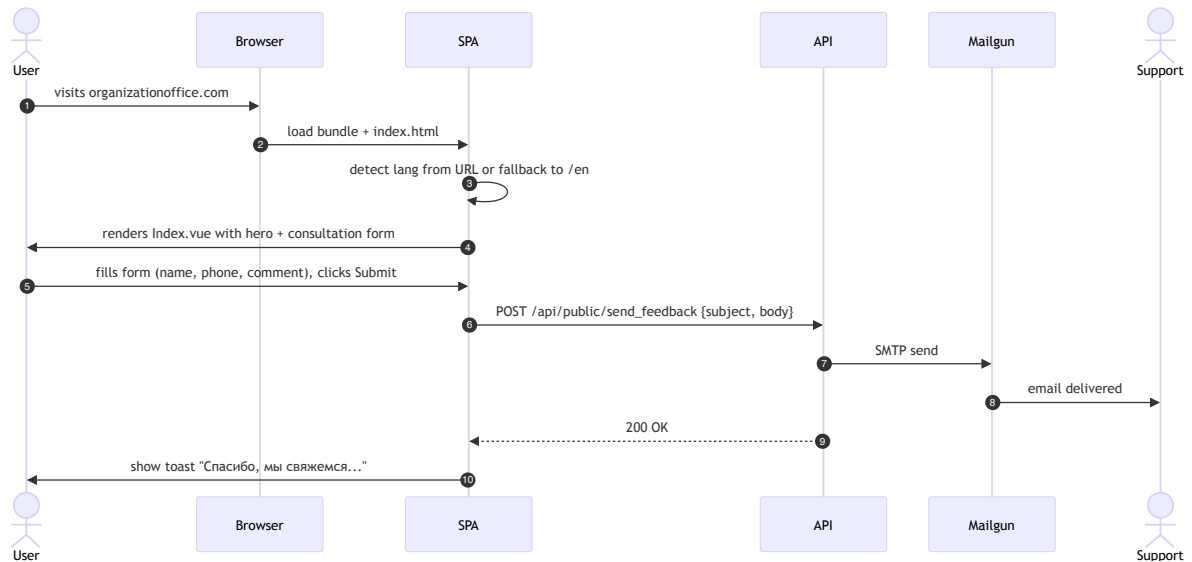
Детали по каждой — [specs/pages.md](https://github.com/project-oo/specs/pages.md).

4. Module Catalogue

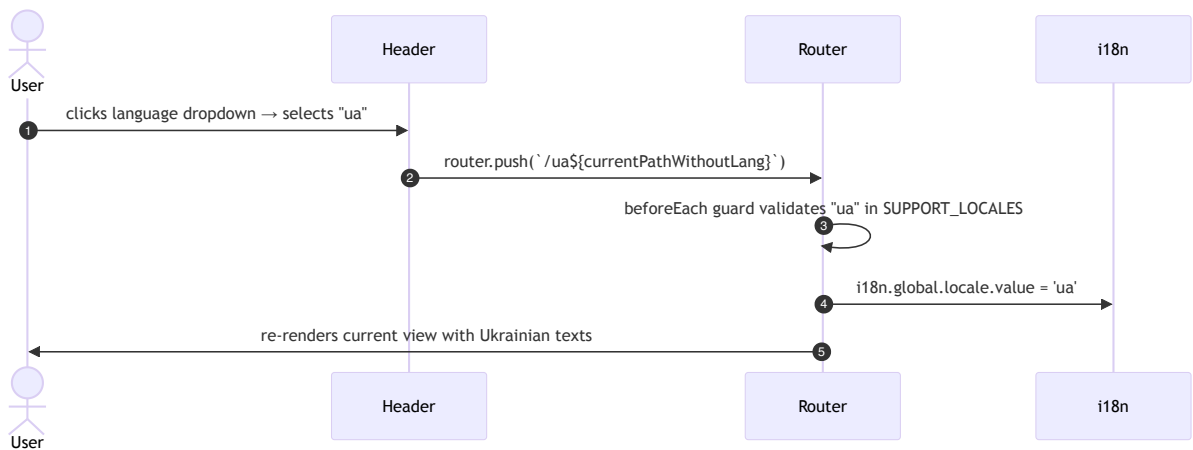
Модуль	Что	Спец
Pages	10 view-страниц с описаниями	specs/pages.md
Routing	Language-prefix роутинг + guards	specs/routing.md
i18n	8 локалей через vue-i18n	specs/i18n.md
Forms	3 формы (callback, consultation, admin)	specs/forms.md
API Integration	axios + send_feedback endpoint	specs/api-integration.md
SEO & PWA	Manifest, favicons, sitemap	specs/seo-pwa.md
Assets	Images, SVG icons, fonts	specs/assets.md

5. User Flows

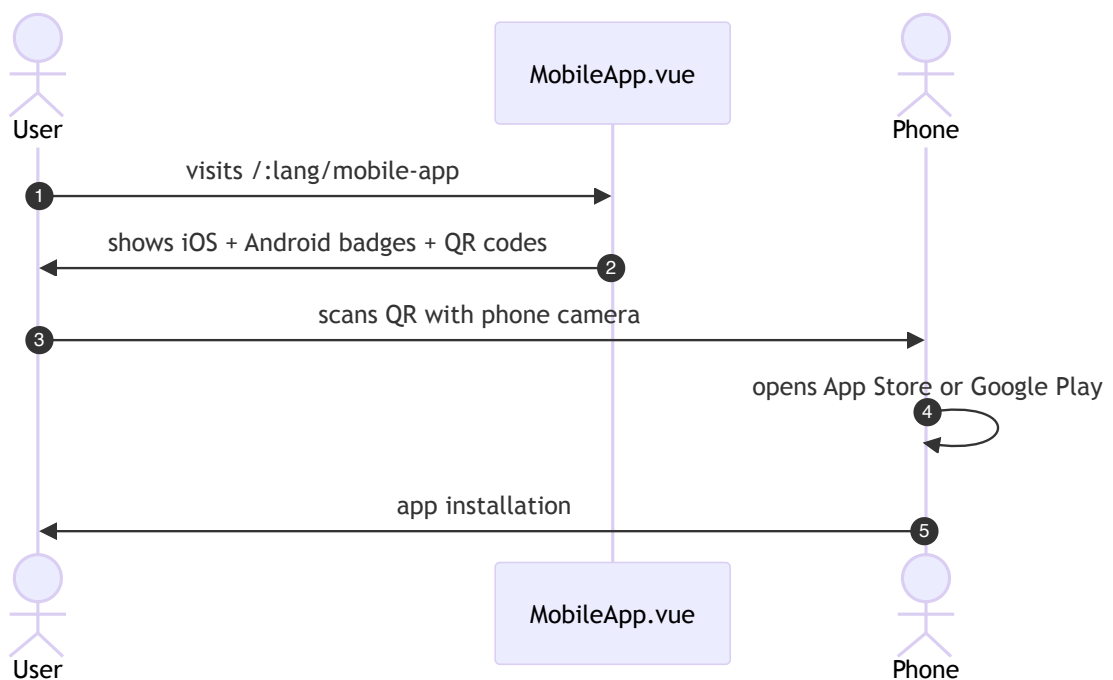
Flow 1: Anonymous visitor lands → fills consultation form



Flow 2: User switches language

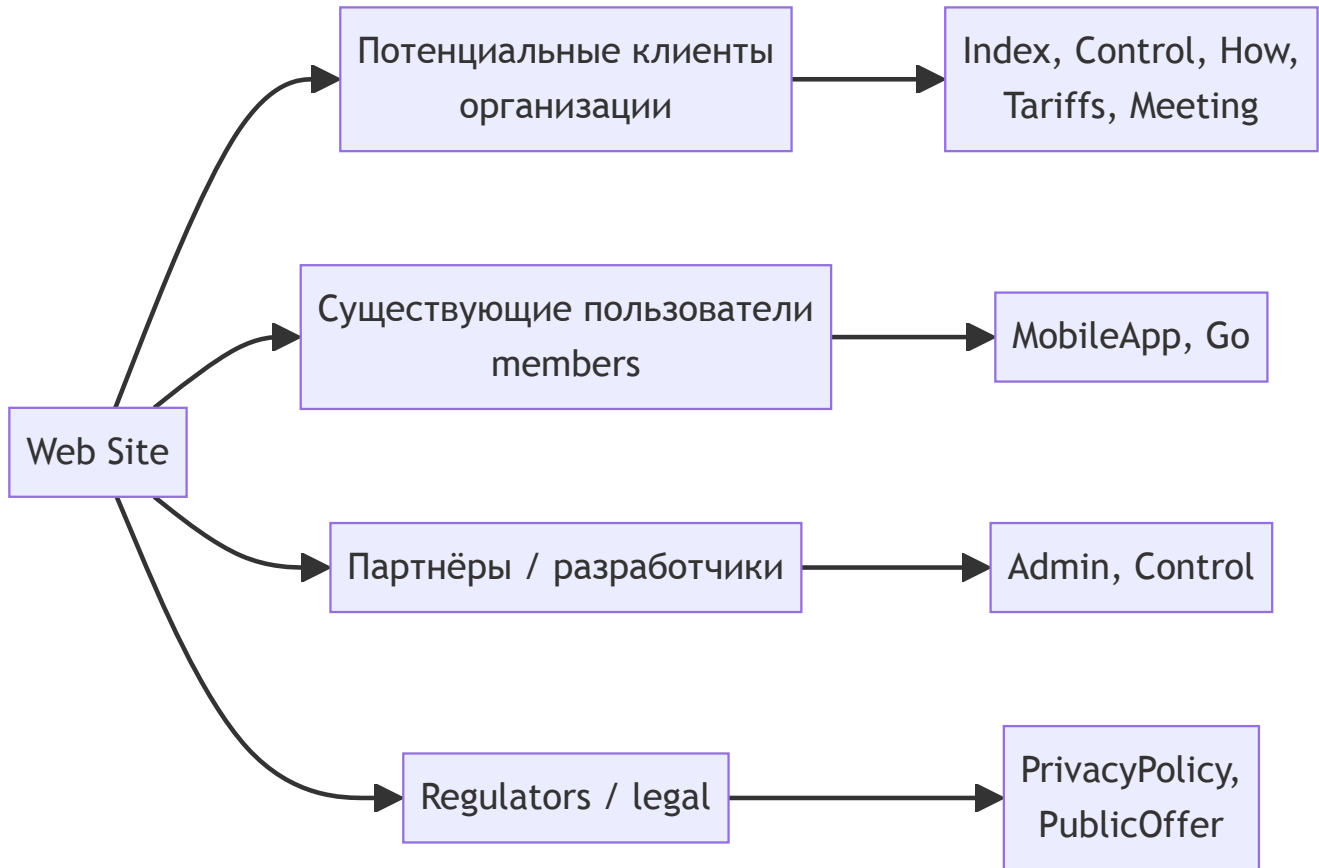


Flow 3: Mobile app promotion → QR scan → app install

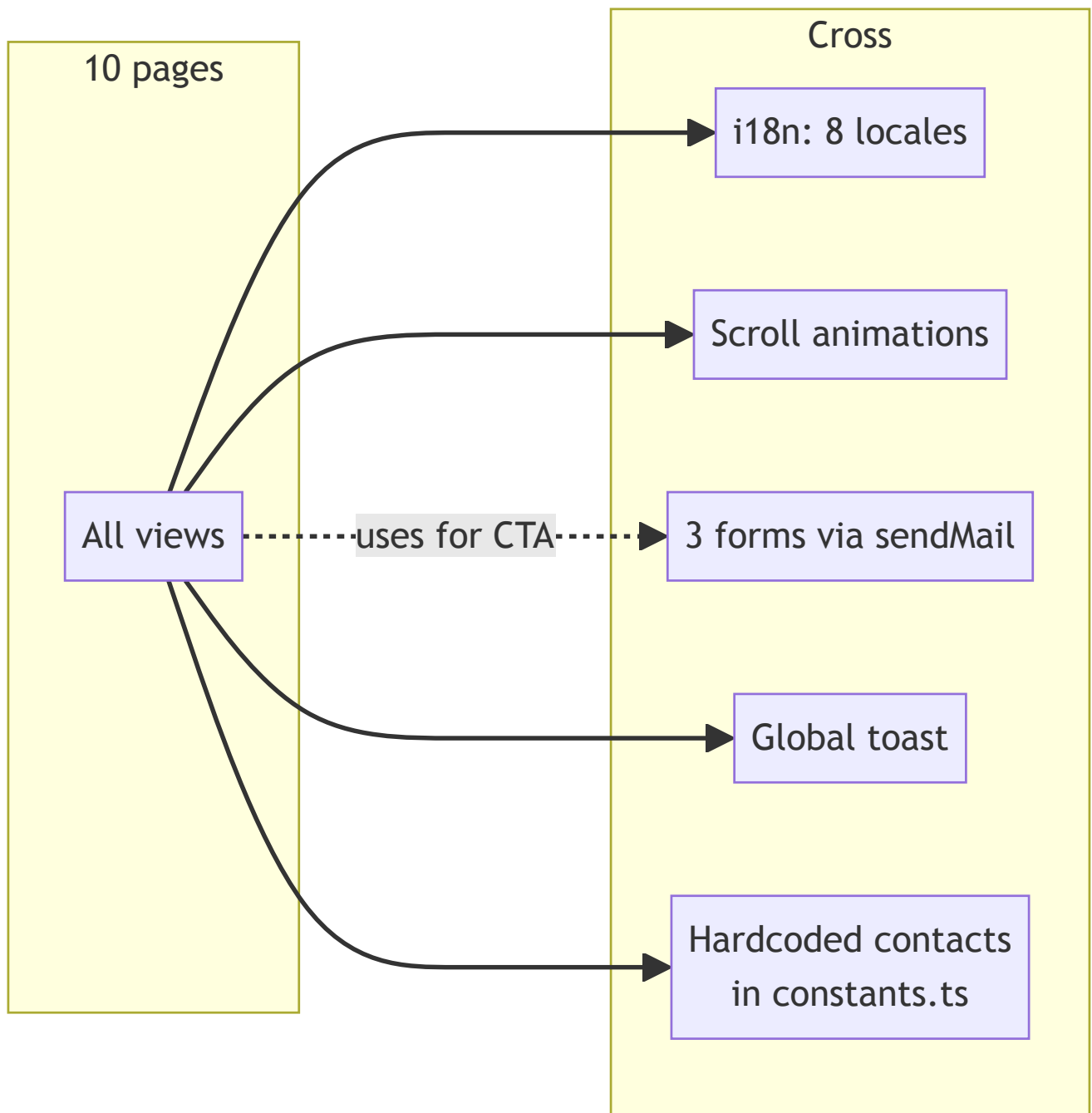


6. Content Strategy

Сайт обслуживает 4 аудитории:



7. Cross-cutting



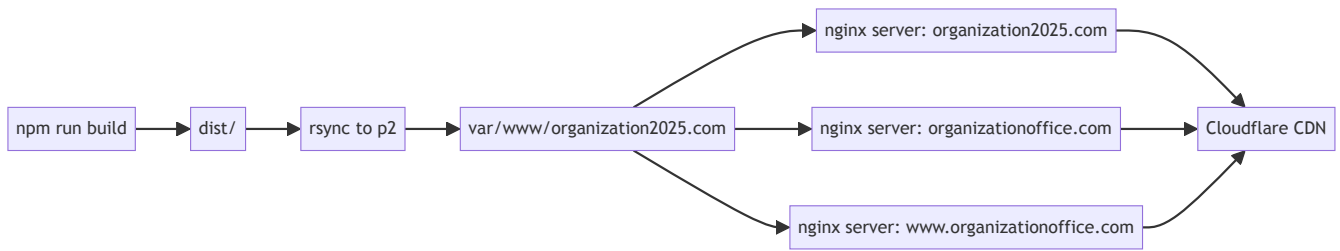
8. Roadmap

Год	Stage	Описание
2025	STAGE 0: Quality	Включить TypeScript strict mode, добавить <code>npm run lint</code> , добавить Vitest unit-тесты
2025	STAGE 1: SEO foundation	<code>@unhead/vue</code> для динамических meta-тегов per page, sitemap.xml generation в build pipeline, robots.txt с правильными правилами
2026	STAGE 2: Performance	Lazy-load 8 lang JSON (только current locale), route-based code splitting, image optimization (WebP/AVIF), preconnect/preload critical resources
2026	STAGE 3: Accessibility	WCAG 2.1 AA compliance: alt-тексты, ARIA labels, focus management, RTL для арабского
2027	STAGE 4: SSR / SSG	Переход на Nuxt 3 или Vite SSG для лучшего SEO (особенно важно для маркетингового сайта)
2027	STAGE 5: PWA enhanced	Service worker для offline, push notifications для уведомлений новостей
2028	STAGE 6: A/B testing	Интеграция с Optimizely / Google Optimize для оптимизации conversion на формах
2029	STAGE 7: CMS	Перенести контент из <code>lang/*.json</code> в headless CMS (Strapi / Sanity) для редактирования без коммита

Known issues (немедленные)

Severity	Issue	Где	Митигация
 HIGH	<code><html lang="ua"></code> хардкодом	<code>index.html</code>	менять динамически по current locale
 HIGH	Нет SEO meta-тегов per page	<code>views/*</code>	<code>@unhead/vue</code>
 MEDIUM	Все 8 lang JSON в bundle	<code>service/i18n.ts</code>	dynamic import
 MEDIUM	Нет sitemap.xml	build	vite-plugin-sitemap
 MEDIUM	Нет robots.txt в <code>public/</code>	<code>public/</code>	добавить
 MEDIUM	RTL для арабского не настроен	<code>App.vue / style.scss</code>	<code>dir="rtl"</code> + CSS-вариант
 MEDIUM	Zapier chatbot sync-loads на любой странице	<code>index.html</code>	defer / условная загрузка
 LOW	<code>strict: false</code> в tsconfig	tsconfig	постепенно strict
 LOW	Нет unit-тестов	—	Vitest
 LOW	Нет <code>lint</code> script в package.json	—	добавить

9. Domains & Deployment



Один build = 3 (и more, если добавятся) домена. Никаких per-domain branding пока не реализовано.

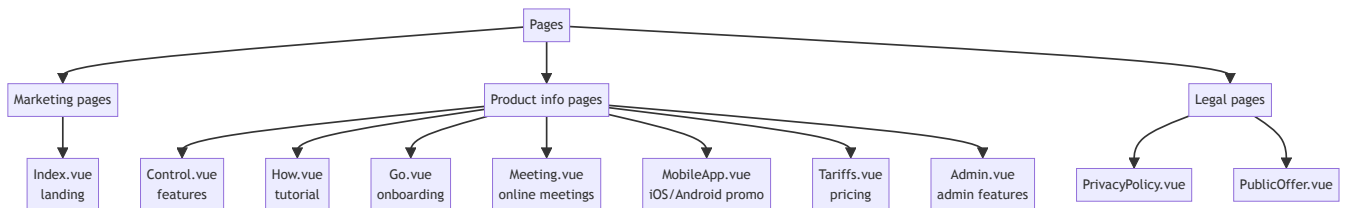
См. также

- [README.md](#) — entry point
- [ARCHITECTURE.md](#) — техническая архитектура
- [SETUP.md](#) — локальная настройка и деплой
- [CONTENT-GUIDE.md](#) — как редактировать контент
- [specs/](#) — модульные спеки

Spec — Pages

10 view-страниц в `src/views/` . Все рутятся через `:lang` prefix.

Structure



Functional Description

Index.vue — Landing

Главная страница, URL `/:lang` .

Структура (typical sections):

- **s_1**: Hero block — заголовок, подзаголовок, CTA, hero-картинка (`/img/index_1_1.png`)
- **s_2**: Stats — KPI с animated count-up
- **s_3**: Trust block — testimonials / партнёры
- **s_4**: Features grid — основные возможности
- **s_5**: How it works (compact)
- **s_6**: CTA + consultation form

Translation keys: `$t('index.s_1.title')` , `$t('index.s_4_1.text')` etc.

Control.vue — Board / Management

URL `/:lang/control` . Описание возможностей управления:

- Управление членами организации
- Документооборот
- Контроль финансов
- Голосования

Картинки: `/img/control_1_1.png` , `/img/control_4_1.png` , ... `/img/control_7_1.png` .

How.vue — Tutorial

URL `/:lang/how` . Step-by-step гайд "Как это работает":

1. Регистрация

2. Настройка организации
3. Добавление членов
4. Запуск процессов
5. Аналитика

Go.vue — Getting Started

URL `/:lang/go` . Onboarding — что делать сразу после регистрации.

Может включать:

- Чек-лист первых шагов
- Демо-видео
- Контакты команды поддержки

Meeting.vue — Online Meetings

URL `/:lang/meeting` . Описание функционала встреч:

- Видеоконференции (если есть)
- Электронная повестка
- Голосование с ЕЦП (через Diia)
- Протоколы

Картинки: `/img/meeting_1_1.png` , `/img/meeting_4_1.png` .

MobileApp.vue — Mobile App Promotion

URL `/:lang/mobile-app` . Реклама мобильного приложения:

- App Store + Google Play badges (через `APPS` из `constants.ts`)
- QR-коды (`/svg/qr2.svg` , `/svg/chat-qr.svg`)
- Screenshot'ы (`/img/mobile-app_1_1.png` , `mobile-app_2_1.png`)
- Список ключевых features

Tariffs.vue — Pricing

URL `/:lang/tariffs` . Тарифные планы.

Структура:

- Tier cards (Basic / Advanced / Premium / Enterprise)
- Каждая карточка: name, price, features list, CTA-кнопка
- Optional: FAQ внизу

Возможно есть form запроса для Enterprise (через `sendMail`).

Admin.vue — Admin Features

URL `/:lang/admin` . Описание возможностей для администраторов организаций.

Может включать форму запроса доступа к админ-панели (для существующих организаций) — через `sendMail`.

PrivacyPolicy.vue — Privacy Policy

URL `/:lang/privacy-policy`. Юридический текст политики конфиденциальности.

Обычно длинный плоский текст с заголовками — HTML-разметка прямо в `template` или через `v-html` с переводами.

PublicOffer.vue — Terms of Service

URL `/:lang/public-offer`. Публичная оферта (украинский контекст: SPA agreement).

Аналогично `PrivacyPolicy` — большой текст с разделами.

Common page elements

Все страницы (кроме `PrivacyPolicy` / `PublicOffer`) используют:

- **Scroll animations** через `scrollAnimate()` — секции появляются при `scroll`'е
- **Responsive layout** — mobile-first через SCSS media queries
- **Lazy load images** (через native `loading="lazy"` на `<img>`)
- **CTA buttons** ведут на формы (Footer callback или page-specific consultation)

Section indexing convention

Внутри каждой страницы секции нумеруются: `s_1`, `s_2`, ... `s_N`. Для блоков внутри секции — `s_N_1`, `s_N_2`. Это удобно для:

- Перевода (легко найти "что это")
- Backend-debug (легко сопоставить URL с переводом)

Pages without translation keys

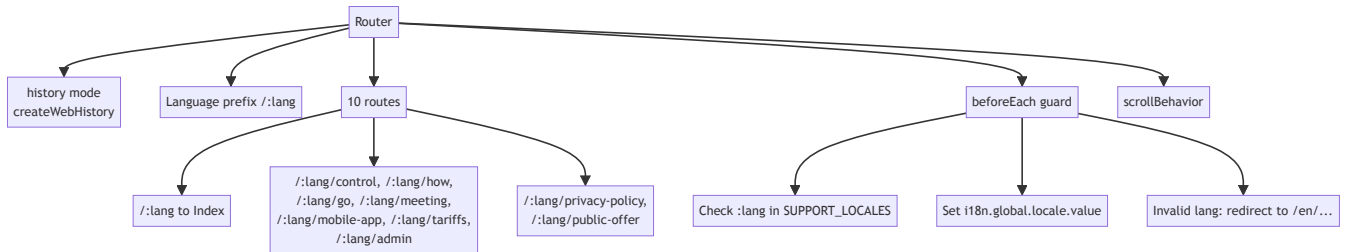
`PrivacyPolicy.vue` и `PublicOffer.vue` могут содержать большие куски HTML inline (потому что юр.тексты не требуют интерактивных элементов). Перевод этих документов — отдельный workflow (часто с юристами).

Known gaps

- **SEO meta-теги per page**: сейчас нет — только глобальный title в `index.html`. Нужно `@unhead/vue` для per-page `<title>` и `<meta description>`.
- **Hero images** для `PrivacyPolicy` / `PublicOffer` отсутствуют → используется generic layout.
- **404 page**: не определена явно — router fallback'ает на `/en`.
- **Performance**: hero-картинки PNG ~200-400 KB каждая → WebP/AVIF сэкономит ~50%.

Spec — Routing

Structure



Functional Description

Mode

`createWebHistory()` — без hash в URL (`/en/control` , не `/#/en/control`). Требуется nginx `try_files $uri /index.html;` для fallback.

Language prefix strategy

Все routes начинаются с `:lang` параметра:

```
const routes: RouteRecordRaw[] = [
  { path: '/:lang', name: 'index', component: Index },
  { path: '/:lang/control', name: 'control', component: Control },
  { path: '/:lang/how', name: 'how', component: How },
  { path: '/:lang/go', name: 'go', component: Go },
  { path: '/:lang/meeting', name: 'meeting', component: Meeting },
  { path: '/:lang/mobile-app', name: 'mobile-app', component: MobileApp },
  { path: '/:lang/tariffs', name: 'tariffs', component: Tariffs },
  { path: '/:lang/admin', name: 'admin', component: Admin },
  { path: '/:lang/privacy-policy', name: 'privacy', component: PrivacyPolicy },
  { path: '/:lang/public-offer', name: 'offer', component: PublicOffer },
]
```

beforeEach guard

```
router.beforeEach((to, from, next) => {
  const lang = to.params.lang as string

  if (!SUPPORT_LOCALES.includes(lang)) {
    // Redirect to default English version of the same path
    const newPath = `/en${to.path.replace(`/${lang}`, '')} || ''`
    next({ path: newPath })
    return
  }

  // Sync i18n with route lang
  i18n.global.locale.value = lang
  next()
})
```

Что происходит:

- `/ua/control` → `ua` ∈ `SUPPORT_LOCALES` → `i18n.locale = 'ua'`, render Control
- `/ru/control` → `ru` ∉ `SUPPORT_LOCALES` → redirect to `/en/control`
- `/control` (без lang) → match `/:lang/control` где `:lang = 'control'` ? **Это не сматчится правильно**, нужно проверить fallback

Edge case: что если пользователь зашёл на корень `/` без lang? Router должен redirect на `/en` или дефолт.

scrollBehavior

```
const router = createRouter({
  // ...
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) return savedPosition // back/forward → restore
    if (to.hash) return { el: to.hash, behavior: 'smooth' } // hash anchor
    return { top: 0 } // default top
  }
})
```

URL examples

URL	Page	Lang
/en	Index	English
/ua	Index	Ukrainian
/ua/control	Control	Ukrainian
/de/tariffs	Tariffs	German
/ar/mobile-app	MobileApp	Arabic
/fr/privacy-policy	PrivacyPolicy	French

Internal links

В компонентах ссылки строятся динамически с текущим locale:

```
<router-link :to="`/${i18n.locale}/control`">
  {{ $t('header.nav1.control') }}
</router-link>
```

Это значит: при смене языка через language switcher все internal-ссылки автоматически перерисовываются с новым prefix.

Language switch implementation

Header.vue :

```
function changeLanguage(newLang: string) {
  const currentPath = route.path.replace(`/${route.params.lang}`, '')
  router.push(`/${newLang}${currentPath || ''}`)
}
```

Логика: вырезать старый lang из URL, подставить новый. View пересоздаётся, i18n переключается через guard.

Permissions

Все страницы — публичные, без auth. Любой visitor может зайти на любую страницу.

Known gaps

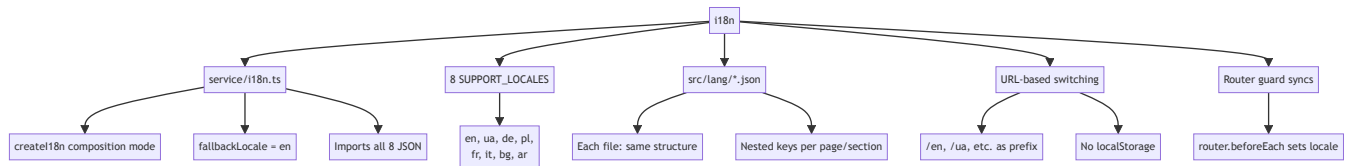
- **Корневой** / (без lang): не определено четко поведение. Должно redirect на /en или на browser locale.
- **404 page**: нет catch-all route. Несуществующая страница в supported lang (например /en/nonexistent) → router показывает blank или фейлит.

```
// Должно быть:  
{ path: '/:lang/:pathMatch(.*)*', component: NotFound }
```

- **Browser language detection:** при первом визите не пытаемся определить язык из `navigator.language` — все redirect'ятся на `/en`.
- **Canonical URLs:** для SEO нужны `<link rel="canonical">` и `<link rel="alternate" hreflang="...">` per language (через `@unhead/vue`).

Spec — i18n

Structure



Functional Description

Setup (`src/service/i18n.ts`)

```

import { createI18n } from 'vue-i18n'
import en from '@/lang/en.json'
import ua from '@/lang/ua.json'
import de from '@/lang/de.json'
import pl from '@/lang/pl.json'
import fr from '@/lang/fr.json'
import it from '@/lang/it.json'
import bg from '@/lang/bg.json'
import ar from '@/lang/ar.json'

export const SUPPORT_LOCALES = ['en', 'ua', 'de', 'pl', 'fr', 'it', 'bg', 'ar']

const i18n = createI18n({
  legacy: false,
  locale: 'en',
  fallbackLocale: 'en',
  messages: { en, ua, de, pl, fr, it, bg, ar }
})

export default i18n

```

Регистрируется в `main.ts` :

```
app.use(i18n)
```

Supported locales

Code	Language	File	Size
en	English	en.json	~25 KB
ua	Ukrainian	ua.json	~30 KB
de	German	de.json	~26 KB
pl	Polish	pl.json	~26 KB
fr	French	fr.json	~26 KB
it	Italian	it.json	~25 KB
bg	Bulgarian	bg.json	~30 KB
ar	Arabic	ar.json	~28 KB

Total: ~215 KB JSON в bundle.

File structure

Все 8 файлов имеют одинаковую иерархическую структуру:

```

{
  "header": {
    "nav1": {
      "control": "Board",
      "members": "Members",
      "meetings": "Meetings"
    },
    "nav2": {
      "tariffs": "Pricing",
      "contacts": "Contacts",
      "admin": "Management"
    },
    "lang": "Language"
  },
  "footer": {
    "callback": "Request a callback",
    "phone": "Phone",
    "email": "Email",
    "working_hours": "Mon-Fri 9:00-18:00",
    "social": "Follow us",
    "text_1": "..."
  },
  "email": {
    "subject_callback": "Callback request",
    "subject_consultation": "Consultation request",
    "subject_admin": "Admin access request",
    "success": "Thank you! We will contact you soon.",
    "error": "Something went wrong. Please try again."
  },
  "index": {
    "s_1": { "title": "...", "subtitle": "...", "cta": "..." },
    "s_2": { "title": "...", "stat_1": "120+", "stat_1_label": "Organizations", ... },
    "s_3": {...},
    ...
  },
  "control": {...},
  "how": {...},
  "go": {...},
  "meeting": {...},
  "mobile_app": {...},
  "tariffs": {...},
  "admin": {...},
  "privacy_policy": {...},
  "public_offer": {...}
}

```

Соглашение по ключам:

- `<page>.s_N` — секция N на странице `<page>`
- `<page>.s_N.title` , `.subtitle` , `.text` , `.cta` — стандартные элементы блока
- `<page>.s_N_M` — sub-block M внутри секции N

Usage

B template (Options API)

```
<h1>{{ $t('index.s_1.title') }}</h1>
<button>{{ $t('index.s_1.cta') }}</button>
```

B script (Composition API)

```
import { useI18n } from 'vue-i18n'
const { t, locale } = useI18n()

const headline = t('index.s_1.title')
```

С интерполяцией

```
{ "greeting": "Hello, {name}!" }
```

```
{{ $t('greeting', { name: visitorName }) }}
```

Language switching

Через router push (см. [routing.md](#)). Шаги:

1. User кликает в Header dropdown → "Ukrainian"
2. Header вызывает `router.push('/ua' + currentPathWithoutLang)`
3. Router guard видит `:lang = 'ua'` → проверяет в `SUPPORT_LOCALES` → set `i18n.global.locale.value = 'ua'`
4. Все компоненты с `$t(...)` автоматически перерисовываются на украинском
5. Все `<router-link :to="/${i18n.locale}/...">` тоже обновляются

Fallback behavior

Если ключ отсутствует в текущем locale:

- vue-i18n использует `fallbackLocale: 'en'` → возвращает английский текст
- В dev mode warning в console
- В prod — silent fallback

No localStorage

В отличие от webadmin / api, **сайт не хранит выбранный язык в localStorage**. Логика:

- При первом посещении → URL без lang → router redirect на `/en`
- Пользователь выбрал украинский → URL стал `/ua/...` → bookmark/refresh сохраняют выбор через URL
- При следующем визите по bookmark — открывается на выбранном языке

Это **намеренно** — URL = single source of truth.

Adding a new language

1. Создать `src/lang/<code>.json` (копия `en.json`, перевести)
2. Добавить импорт и в `messages` в `service/i18n.ts`
3. Добавить в `SUPPORT_LOCALES`
4. Добавить в language dropdown в `Header.vue`
5. (опц.) Если RTL (например, `he`, `ar`) — настроить `<html dir="rtl">` динамически

Adding a new translation key

1. Открыть **все 8** JSON-файлов
2. Добавить ключ в одно и то же место в каждом
3. Перевести
4. Использовать `$t('new.key')` в компоненте

Tip: `grep -r '"new_key":' src/lang/` — найти/убедиться что добавлено везде.

Number / Date formatting

`vue-i18n` умеет форматировать `per-locale`, но в текущей реализации **НЕ используется**. На сайте мало числовой/датовой информации — в основном маркетинговый текст.

Если потребуется (например, формат цен в тарифах) — использовать `$n(value, format, locale)` или встроенный `Intl.NumberFormat(locale).format(value)`.

RTL для арабского

Сейчас НЕ настроено. Арабский (`ar`) текст отображается, но layout остаётся LTR.

Чтобы исправить — добавить в `App.vue`:

```
import { watch } from 'vue'
import { useI18n } from 'vue-i18n'

const { locale } = useI18n()
watch(locale, (newLang) => {
  document.documentElement.dir = newLang === 'ar' ? 'rtl' : 'ltr'
  document.documentElement.lang = newLang
}, { immediate: true })
```

И в `style.scss` — RTL-вариант для key-стилей (или использовать logical properties `padding-inline-start` вместо `padding-left`).

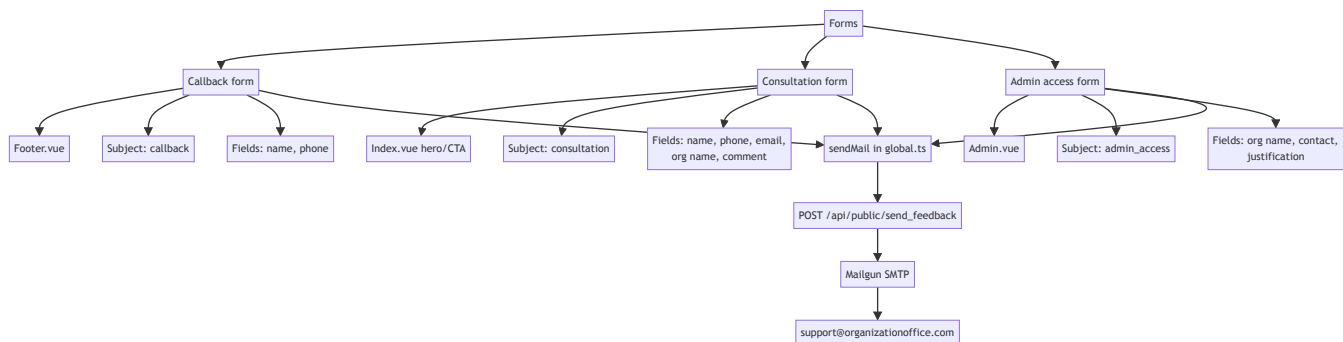
Known gaps

- `<html lang>` хардкодом на `ua` в `index.html` → не синхронизировано с current locale
- RTL для арабского не настроено
- Все 8 locale в bundle — можно lazy-load
- Sync between languages — manual edit всех 8 файлов; tooling нет
- Missing keys telemetry — нет в prod
- Pluralization не используется — но есть в vue-i18n как `$tc`

Spec — Forms

3 формы обратной связи на сайте — все отправляют через единый endpoint `/api/public/send_feedback`.

Structure



Functional Description

Callback form (Footer.vue)

Мини-форма в footer'е, доступна со всех страниц.

Поле	Required	Validation
Name	✓	non-empty
Phone	✓	not-strict format

Submit → toast "Спасибо, мы свяжемся с вами" → reset формы.

Consultation form (Index.vue)

Основная форма на лендинге — для запроса демо/консультации.

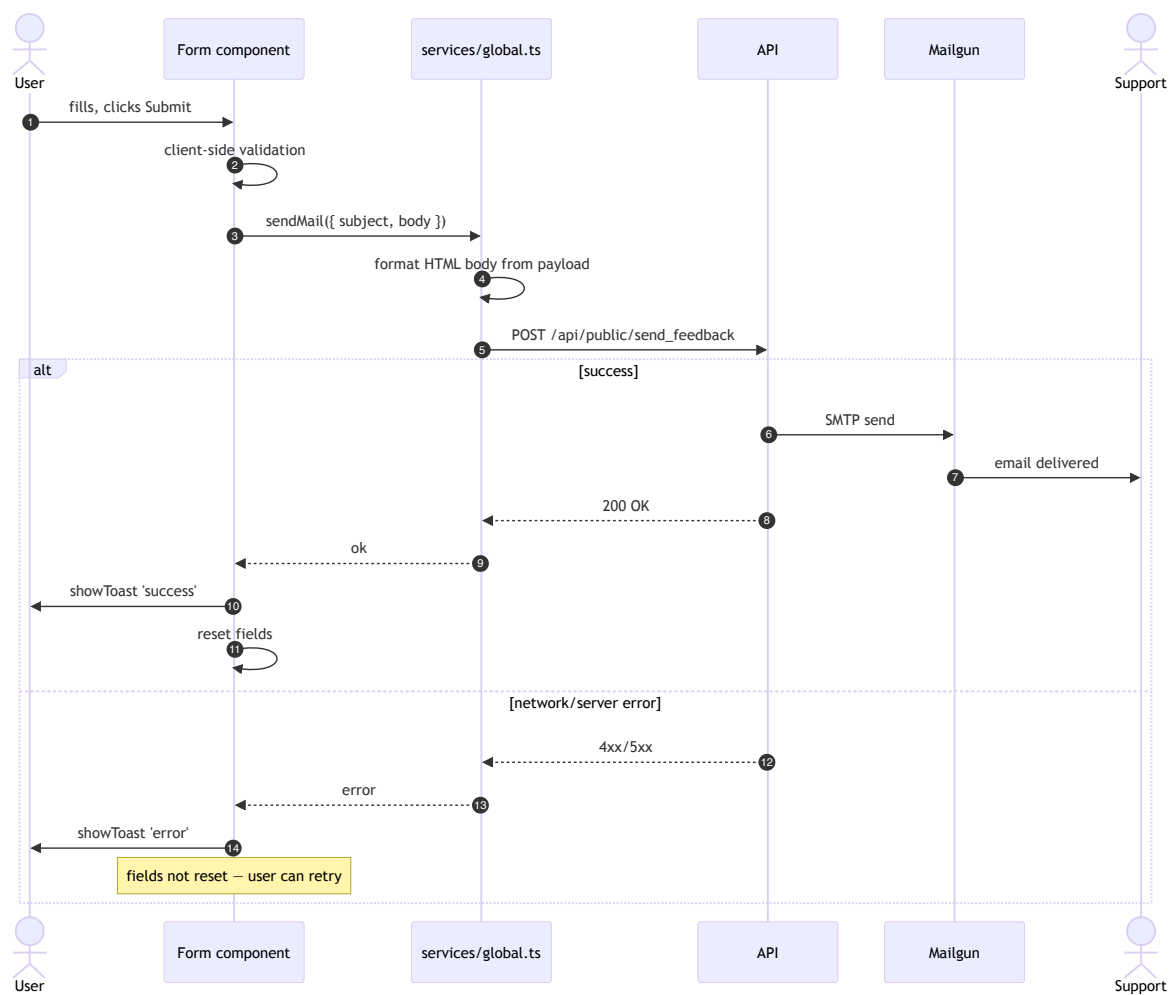
Поле	Required	Validation
Name	✓	non-empty
Phone	✓	not-strict
Email	optional	RFC email if filled
Organization name	optional	non-empty if filled
Message / comment	optional	textarea

Admin access form (Admin.vue)

Для существующих администраторов организаций, потерявших доступ или запрашивающих расширенные права.

Поле	Required	Validation
Organization name	✓	non-empty
Contact name	✓	non-empty
Contact email/phone	✓	non-empty
Justification / details	✓	textarea

Submission flow



sendMail() — глобальный helper

src/service/global.ts :

```
import axios from 'axios'
import i18n from '@service/i18n'

export async function sendMail(payload: {
  subject: string
  body: string
  // или: name, phone, email, comment — формат уточняется
}) {
  try {
    await axios.post('/api/public/send_feedback', payload)
    showToast(i18n.global.t('email.success'), 'success')
    return true
  } catch (error) {
    showToast(i18n.global.t('email.error'), 'error')
    return false
  }
}
```

Регистрируется как global property в `main.ts` :

```
app.config.globalProperties.$sendMail = sendMail
```

Использование в компонентах: `this.$sendMail({ subject, body })` .

Email body format

Backend получает в payload поле `body` с HTML-форматированным содержимым формы. Пример:

```
<h2>Callback request</h2>
<p><b>Name:</b> Иван Петренко</p>
<p><b>Phone:</b> +380501234567</p>
<p><b>Submitted at:</b> 2025-10-15 14:30 UTC</p>
<p><b>Source:</b> https://organizationoffice.com/ua</p>
```

Backend (`VerificationService.sendFeedback`) пересылает на `spring.mail.support` (= `support@organizationoffice.com`).

Anti-spam

Сейчас: на сайте нет CAPTCHA / honeypot / rate-limiting на стороне frontend. Backend должен иметь:

- Rate limit per IP (например, ≤5 запросов/минуту на `/api/public/send_feedback`)
- Опционально — honeypot field (hidden field, который боты заполняют, а humans — нет)
- Опционально — Cloudflare Turnstile (бесплатная альтернатива reCAPTCHA)

Roadmap: добавить honeypot + Turnstile, см. SPECIFICATION.md STAGE 1.

Validation

Client-side: minimal — required check + phone format hint. **HE** strict validation (например, не используется `vee-validate`).

Server-side: бэкенд должен sanitize/validate (особенно для HTML в body — XSS-риск при пересылке email).

i18n

Тексты форм (labels, placeholders, button labels, error/success messages) хранятся в `lang/*.json` под ключами `email.*`:

```
{
  "email": {
    "subject_callback": "Callback request",
    "subject_consultation": "Consultation request",
    "subject_admin": "Admin access request",
    "success": "Thank you! We will contact you within 24 hours.",
    "error": "Something went wrong. Please try again or call us directly.",
    "field_name": "Name",
    "field_phone": "Phone",
    "field_email": "Email",
    "field_org": "Organization",
    "field_comment": "Comment",
    "btn_submit": "Submit"
  }
}
```

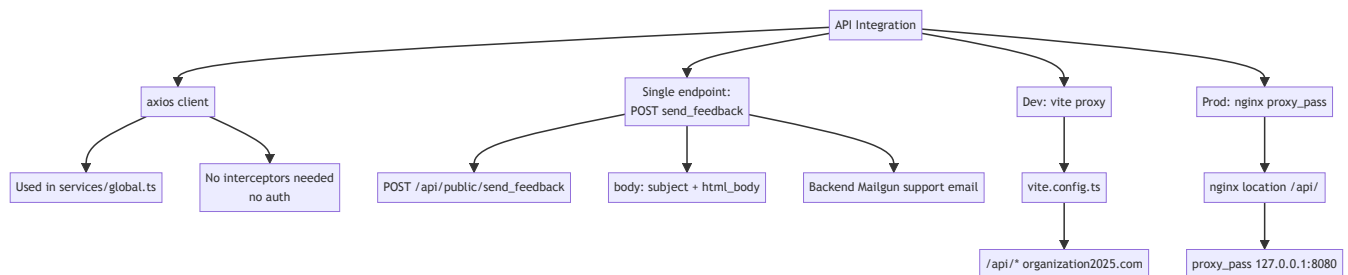
Known gaps

- **Нет CAPTCHA / honeypot** — формы открыты для ботов
- **Client-side validation минимальная** — нет инлайн-ошибок per-field
- **sendMail не показывает loading state** — пользователь может кликнуть Submit многократно
- **Нет idempotency** — повторный submit отправит email ещё раз
- **Нет form analytics** (например, какая % submissions vs visits)
- **Backend error sanitization**: если backend возвращает error message, он не локализован — на frontend всегда `email.error` (generic).

Spec — API Integration

Минимальная интеграция: один endpoint для форм + проксирование статики через nginx.

Structure



Functional Description

axios usage

В отличие от `webadmin`, здесь нет глобального `api` instance — `axios.post(...)` вызывается напрямую в `services/global.ts`:

```
import axios from 'axios'

export async function sendMail(payload) {
  try {
    await axios.post('/api/public/send_feedback', {
      subject: payload.subject,
      body: payload.body
    })
    showToast(i18n.global.t('email.success'))
    return true
  } catch (e) {
    showToast(i18n.global.t('email.error'))
    return false
  }
}
```

Нет authentication, нет interceptor — endpoint публичный.

The only endpoint

`POST /api/public/send_feedback` — единственный, который сайт использует.

Request:

```
{
  "subject": "Callback request",
  "body": "<h2>...</h2><p>Name: ...</p>"
}
```

Response: 200 OK на успех, 4xx/5xx на ошибку.

Backend implementation

См. [api/docs/specs/integrations.md](#). Кратко:

- Spring Boot контроллер `PublicController.send_feedback`
- Использует `VerificationService.sendFeedback`
- Отправляет через `JavaMailSender` (Mailgun SMTP)
- Адрес назначения — `spring.mail.support` (`support@organizationoffice.com`)

Dev proxy (vite.config.ts)

```
server: {
  proxy: {
    '/api': {
      target: 'https://organization2025.com',
      changeOrigin: true
    }
  }
}
```

В dev все `/api/*` идут на продакшен бэкенд. Для разработки против локального API — поменять target на `http://localhost:8080`.

Production routing

nginx:

```
location /api/ {
  proxy_pass http://127.0.0.1:8080/;
  proxy_set_header Host $host;
}
```

Тот же бэкенд (Spring Boot на 8080) обслуживает и `webadmin`, и `mobile API`, и формы публичного сайта.

XML data: `/public/xml/rgo.xml`

Файл с публичными реестровыми данными общественных организаций (Украина). Не загружается через API — это статика, парсится клиентом через `fast-xml-parser`:

```
import { XMLParser } from 'fast-xml-parser'

const response = await fetch('/xml/rgo.xml')
const text = await response.text()
const data = new XMLParser().parse(text)
```

Если используется в одном-двух местах — стоит lazy-load.

Static assets

Все из public/ раздаются nginx как статика: img/, svg/, xml/, favicons, manifest. В browser cache живут долго.

CORS

Не нужен — same-origin в prod. В dev vite proxy маскирует cross-origin.

Error handling

`sendMail` — try/catch + generic toast. Backend message игнорируется на frontend; пользователь видит generic "Something went wrong".

Permissions

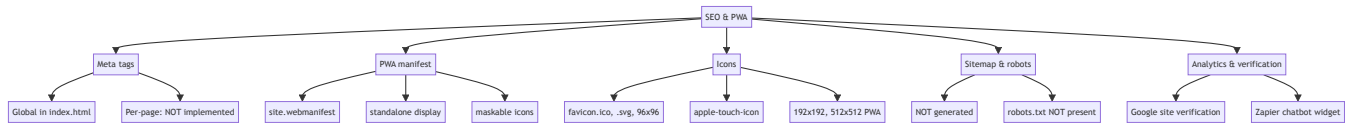
Endpoint публичный — без JWT. Backend должен иметь rate-limiting per IP + опционально honeypot/CAPTCHA на форме.

Known gaps

- No request idempotency — повторный submit отправит email повторно
- No retry logic — single attempt
- Generic errors only — не показываем server-side message
- No timeout — axios default (нет timeout вообще)
- No request cancellation при unmount компонента
- No analytics на API-вызовы

Spec — SEO & PWA

Structure



Functional Description

Global meta tags (index.html)

```

<!doctype html>
<html lang="ua"> <!-- 🟡 hardcoded – should be dynamic -->
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" type="image/svg+xml" href="/favicon.svg">
  <link rel="icon" type="image/png" sizes="96x96" href="/favicon-96x96.png">
  <link rel="apple-touch-icon" sizes="180x180" href="/apple-touch-icon.png">
  <link rel="manifest" href="/site.webmanifest">
  <meta name="google-site-verification" content="...">
  <title>Organization2025</title>
</head>

```

Текущее состояние:

- ✅ Charset / viewport / favicons / manifest
- ✅ Google site verification
- 🟡 `<html lang="ua">` хардкодом → не синхронизировано с current locale
- ❌ Per-page `<title>` — везде одинаковый "Organization2025"
- ❌ `<meta name="description">` — отсутствует
- ❌ Open Graph / Twitter Card meta — отсутствует (плохое preview при шеринге в соцсетях)
- ❌ `<link rel="canonical">` — нет (плохо для duplicate-content)
- ❌ `<link rel="alternate" hreflang="...">` — нет (для multi-language SEO)

PWA Manifest (site.webmanifest)

```
{
  "name": "MyWebSite",
  "short_name": "MySite",
  "icons": [
    {
      "src": "/web-app-manifest-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable"
    },
    {
      "src": "/web-app-manifest-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable"
    }
  ],
  "theme_color": "#ffffff",
  "background_color": "#ffffff",
  "display": "standalone"
}
```

Issues:

- Имя "MyWebSite" / "MySite" — placeholder. Должно быть "OrganizationOffice" или "Organization2025"
- Нет `start_url` (по умолчанию /)
- Нет `scope` (по умолчанию домен origin'a)
- Нет `description`
- Нет `lang`
- Нет `categories` (для discovery в PWA stores)
- `purpose: maskable` правильно, но желательно добавить `any` версию (для устройств без masking)

Favicons

Файл	Размер	Назначение
<code>favicon.ico</code>	15 KB	Legacy browsers
<code>favicon.svg</code>	scalable	Modern browsers (auto-scaling)
<code>favicon-96x96.png</code>	96×96	Standard PNG
<code>apple-touch-icon.png</code>	180×180	iOS Safari "Add to home"
<code>web-app-manifest-192x192.png</code>	192×192	PWA install icon (Android)
<code>web-app-manifest-512x512.png</code>	512×512	PWA splash / large icon

Полный комплект. При смене брендинга нужно регенерировать всё (через realfaviconsgenerator.net).

Sitemap

✗ **НЕТ** — `sitemap.xml` не генерируется и не лежит в `public/`.

Должен быть для:

- Submission в Google Search Console
- Crawl-friendly навигация по всем 10 страницам × 8 языков = 80 URL

Решение: vite plugin типа [vite-plugin-sitemap](#):

```
// vite.config.ts
import Sitemap from 'vite-plugin-sitemap'

export default defineConfig({
  plugins: [
    vue(),
    Sitemap({
      hostname: 'https://organizationoffice.com',
      dynamicRoutes: [
        '/en', '/en/control', '/en/how', /* ... */,
        '/ua', '/ua/control', /* ... */,
        // ... × 8 locales × 10 pages = 80 URLs
      ]
    })
  ]
})
```

Получим `dist/sitemap.xml` при build.

robots.txt

✗ **НЕТ** в `public/`.

Должен быть (минимум):

```
User-agent: *
Allow: /
Disallow: /api/

Sitemap: https://organizationoffice.com/sitemap.xml
```

Open Graph / Twitter Card

✗ **НЕТ.** Когда ссылка на сайт шарится в Telegram / Facebook / Twitter — preview без картинки и описания.

Должно быть (per page, через `@unhead/vue`):

```
<meta property="og:title" content="OrganizationOffice – Manage your community">
<meta property="og:description" content="...">
<meta property="og:image" content="https://organizationoffice.com/img/index_1_1.png">
<meta property="og:url" content="https://organizationoffice.com/en">
<meta property="og:type" content="website">

<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="...">
<meta name="twitter:image" content="...">
```

Google Site Verification

✓ Присутствует в `index.html` :

```
<meta name="google-site-verification" content="...">
```

Это значит — сайт уже в Google Search Console. Имеет смысл периодически проверять там Coverage / Issues.

Zapier Chatbot widget

```
<zapier-interfaces-chatbot-embed chatbot-id="cmftsrv4001a10558q2960z1"></zapier-interfaces-
```

Скрипт загружается синхронно — добавляет ~100-200 KB JS + ~500 ms к TTI. Альтернативы:

- Lazy-load по интеракции (click on "?" иконку)
- Defer (`defer` attribute или dynamic `<script>` insertion после `DOMContentLoaded`)

Performance budget (proposed)

Метрика	Target	Текущее (estimate)
Lighthouse Performance	≥ 90	? — нужно проверить
LCP	< 2.5s	?
FID	< 100ms	?
CLS	< 0.1	?
Bundle JS (gzip)	< 200 KB	~150-200 KB
Bundle CSS (gzip)	< 30 KB	~20-30 KB
Total page weight	< 1 MB	?

Рекомендация: запустить Lighthouse audit + WebPageTest, добавить в CI.

Recommended SEO improvements (приоритеты)

1. `@unhead/vue` для **per-page meta** — добавит title, description, OG tags
2. **Sitemap + robots.txt** — basic SEO compliance
3. **Dynamic `<html lang>` + hreflang** — для multi-language SEO
4. **Image optimization** — WebP / AVIF, responsive images через `<picture>`
5. **Lazy-load Zapier widget** — улучшит Lighthouse Performance
6. **SSR / SSG** — переход на Nuxt 3 или vite-ssg даст лучший SEO crawl (для маркетинга это критично)

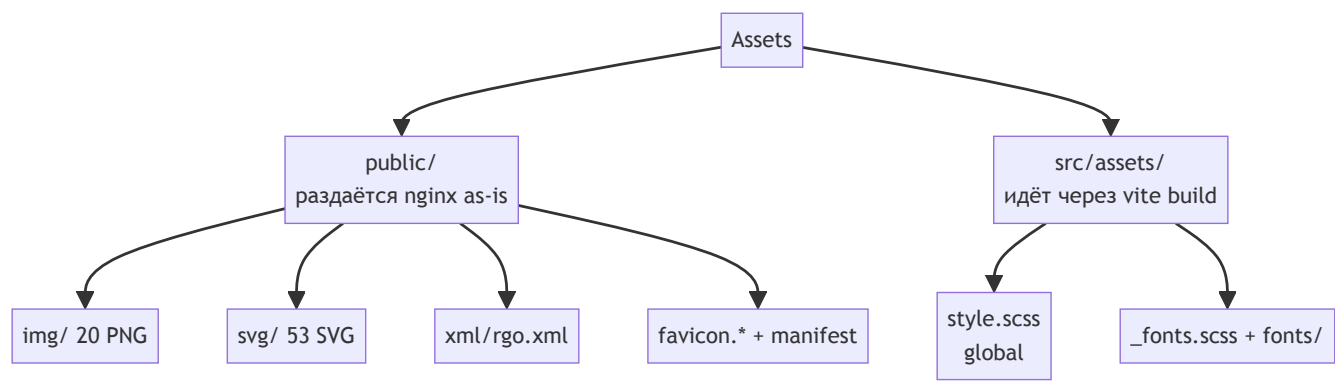
Known gaps

- 🟡 PWA manifest имеет placeholder name "MyWebSite"
- 🟡 Нет per-page meta (title, description, OG)
- 🟡 Нет sitemap.xml
- 🟡 Нет robots.txt
- 🟡 Hardcoded `<html lang="ua">`
- 🟡 Нет hreflang для multi-language
- 🟡 Service worker / offline mode не настроен
- 🟢 Zapier chatbot sync-load → 100-200ms TTI penalty

Спец — Assets

Изображения, иконки, шрифты, стили.

Structure



public/ — раздаётся nginx as-is

public/img/ — PNG картинки (20 файлов)

Категория	Файлы	Назначение
Hero images	index_1_1.png , control_1_1.png , meeting_1_1.png , mobile-app_1_1.png	Большие картинки в первом блоке (s_1) каждой страницы
Feature blocks	control_4_1.png , control_5_1.png , control_6_1.png , control_7_1.png , index_4_1.png , index_5_1.png , index_6_1.png , meeting_4_1.png , mobile-app_2_1.png	Иллюстрации внутри секций
Team	team_1.png .. team_7.png	7 портретов команды (вероятно на Index или About)

Соглашение по именам: `<page>_<section>_<index>.png` .

Формат — PNG. **Roadmap:** миграция на WebP / AVIF (экономит 30-50% размера).

public/svg/ — Иконки и декор (53 файла)

Brand

- `logo.svg` , `logo-sm.svg` — главный логотип + small variant

Social media (по 2 варианта — для dark и white backgrounds)

- Instagram: `in.svg` , `in-w.svg`
- YouTube: `you.svg` , `you-w.svg` , `you-b.svg`
- TikTok: `tt.svg` , `tt-w.svg`
- Telegram: `tg.svg` , `tg-b.svg`
- Facebook: `f.svg` , `f-w.svg`
- WhatsApp: `wa.svg` , `wa-w.svg`
- LinkedIn: `linkedin.svg` , `linkedin-w.svg`

App stores

- `apple.svg` , `apple-w.svg` — App Store
- `googleplay.svg` — Google Play

UI icons (descriptive names)

- `check.svg` , `check-2.svg` , `check-3.svg` — checkmarks (3 варианта)
- `arrow-right.svg` , `arrow-right-2.svg` , `arrow-right-3.svg` — стрелки
- `like.svg` , `click.svg` , `download.svg` , `accept.svg`
- `communication.svg` , `customer.svg` , `users.svg`
- `documents.svg` , `file.svg` , `registration-form.svg`
- `video-call.svg` , `online-meeting.svg`
- `hash.svg` , `list-search.svg`

Decorative

- `triangle.svg` , `triangle-2.svg` , `triangle-3.svg` — геометрические формы
- `bg-1.svg` , `bg-2.svg` , `bg-3.svg` , `bg-4.svg` — background patterns
- `chat-qr.svg` , `qr2.svg` — QR-коды (вероятно для Telegram и mobile app)

Соглашение: `-w` суффикс = white version, `-b` = black/blue.

public/xml/

- `rgo.xml` — Public org registry data (Україна, Реєстр громадських об'єднань). Парсится через `fast-xml-parser` если используется (нужно проверить, где именно).

public/ — favicons & PWA

Файл	Размер	Назначение
<code>favicon.ico</code>	15 KB	Legacy browsers
<code>favicon.svg</code>	scalable	Modern, supports theming
<code>favicon-96x96.png</code>	96×96	Standard PNG
<code>apple-touch-icon.png</code>	180×180	iOS home screen
<code>web-app-manifest-192x192.png</code>	192×192	PWA Android
<code>web-app-manifest-512x512.png</code>	512×512	PWA splash
<code>site.webmanifest</code>	—	PWA manifest

src/assets/ — обрабатывается vite

style.scss

Главный стиль (импортируется в `main.ts`). Содержит:

Цветовая палитра

```
// Backgrounds
$bg-main:      #DBD6DE; // светло-фиолетовый — главный фон
$bg-secondary: white;
$bg-2:         #2E80B7; // синий акцент
$bg-3:         black;
$bg-4, $bg-5, $bg-6, $bg-7, $bg-8: #...; // pastel variations

// Text colors
$text-main, $text-light, $text-accent, $text-dark
```

Глобальные resets

* { `box-sizing: border-box;` }, нормализация margin/padding, font-family.

Utility classes

Flex utilities (`.flex` , `.flex-center` , `.flex-between`), spacing (`.mt-4` , `.p-8`), text alignment, shadows, transitions, animation keyframes (для scroll-reveal).

_fonts.scss

`@font-face` декларации для **Roboto Condensed** (3 веса: Regular, Medium, Bold).

```
@font-face {
  font-family: 'Roboto Condensed';
  src: url('../fonts/RobotoCondensed/RobotoCondensed-Regular.ttf') format('truetype');
  font-weight: 400;
  font-display: swap;
}
// + Medium (500) и Bold (700)
```

fonts/RobotoCondensed/

TTF файлы шрифтов:

- RobotoCondensed-Regular.ttf
- RobotoCondensed-Medium.ttf
- RobotoCondensed-Bold.ttf

Roadmap: конвертировать в WOFF2 (10x меньше, поддерживается всеми современными браузерами). Использовать `font-display: swap` для perf.

Asset pipeline differences

public/ vs src/assets/

Источник	Обработка	URL в production
public/img/foo.png	копируется в <code>dist/</code> as-is	/img/foo.png
public/svg/logo.svg	копируется в <code>dist/</code> as-is	/svg/logo.svg
src/assets/bg.png (если был бы)	hash-suffixed, бандлится	/assets/bg-a1b2c3d4.png
src/assets/style.scss	компилируется в CSS	/assets/index-xyz.css

Правило: статика, которая ссылается напрямую через URL (``) — в `public/` . Стили и шрифты, которые импортируются через `import './style.scss'` — в `src/assets/` .

CDN / caching

В production раздаются через nginx → Cloudflare CDN. Cache-Control headers должны быть:

- HTML / SPA index — `Cache-Control: no-cache` (всегда свежий)
- JS/CSS с hash в имени — `Cache-Control: max-age=31536000, immutable` (год)
- Images/SVG — `Cache-Control: max-age=2592000` (месяц)
- favicons / manifest — `Cache-Control: max-age=2592000`

Конфиг для nginx нужно проверить — может быть default.

Optimization checklist

- ☐ PNG → WebP/AVIF (через `npx @squoosh/cli` или `image-min` плагин для vite)
- ☐ SVG минимизированы (через [SVGOMG](#))
- ☐ TTF → WOFF2 для шрифтов
- ☐ `font-display: swap` чтобы текст показывался сразу
- ☐ Critical CSS inline в `<head>` (`vite-plugin-html-inline`)
- ☐ Lazy-load для below-the-fold images (`loading="lazy"`)
- ☐ Preconnect для CDN / Zapier

Known gaps

- PNG не оптимизированы / не сконвертированы в WebP
- SVG могут содержать metadata (Inkscape, Illustrator) — экономия 10-30% после оптимизации
- Шрифты в TTF (не WOFF2) — лишний download size
- Нет proactive preload для критических ресурсов
- Нет responsive images (`<picture>` + multiple sizes)